CHAPTER 3: REST APIS

LEARNING OBJECTIVES

- Be able to explain the basic principles of REST.
- Be able to design REST interfaces and implement them with Node.js/express.
- Be able to explain the advantages and disadvantages of REST.

3.1 WHAT IS REST?

- REST stands for Representational State Transfer and goes back to Roy Fielding's dissertation from the year 2000.
- REST is an architectural style used for designing networked applications. It relies on a stateless, client-server, cacheable communications protocol — the HTTP protocol is almost always used.

BASIC PRINCIPLES OF REST:

- **Resources** (data or services that clients interact with, e.g. an order):
  - Unique identification of resources via URLs
  - Resource linking: Resources are linked to other resources via URLs (e.g. order has customer)
- HTTP **standard methods**:
  - Servers are called using the basic HTTP commands GET, PUT, POST, DELETE, PATCH
- Multiple **Representations** of a Resource:
  - Different representations (text, XML, JSON, HTML) can be used for data exchange
- **Stateless** architecture:
  - Communication with the HTTP server is stateless (stateless http server), making the server highly adaptable to increasing user numbers (scalability).

IDENTIFICATION OF RESOURCES

Each resource is uniquely identified via its URL.

Examples:

URL	Interpretation
http://example.com/customers/1234	Customer with ID 1234
http://example.com/orders/2007/10/776654	Order of Oct 2007 with ID 776654
http://example.com/products/4554	Product with ID 4554
http://example.com/processes/sal-increase-234	salary increase for quarters 2, 3 and 4

RESOURCE-LINKAGE

- Data models consisting of objects with references to other objects are translated into resource networks
- A resource can have any number of links to other resources

Example of an order resource

Order 1234: 23 products with id 44533 ordered by customer with Id 4711

```
{
    "orderId":      1234,
    "amount":       23,
    "productRef":   "http://example.com/products/44533",
    "customerRef":   "http://example.com/customers/4711"
}
```

EXERCISE: RESOURCE-LINKAGE

Given the following resource:

```
{
    "orderId":      1234,
    "amount":       23,
    "productRef":   "http://example.com/products/44533",
    "customerRef":  "http://example.com/customers/4711"
}
```

Give a corresponding UML class model for the above example!

STANDARD METHODS

Operations of a REST service refer to a resource and can be called using the means offered by HTTP. The typical HTTP methods GET, POST, PUT, PATCH and DELETE have a standard interpretation in this context:

Method	Interpretation
GET	retrieve information about a resource
POST	submit data to the server, often used for creating new resources
PUT	update resource or create sub-resource
DELETE	delete a resource
PATCH	partially update a resource - only send changed parts

Examples for standard method calls: (server-prefix omitted)

method	path	body	semantics
GET	order		return all orders
GET	order/1		return order with ID 1
POST	order	{ "product": "milk", "amount": 2 }	create new order based on body data
PUT	order/1	{ "product": "milk", "amount": 13 }	update order with ID 1 based on body data
DELETE	order/1		delete order with ID 1
PATCH	order/1	{ "amount": 13 }	partially update order with ID 1 based on body data

EXERCISE: STANDARD METHODS 1

EXERCISE: HTTP STANDARD METHODS

What are valid standard methods?

- [ ] POST
- [ ] GET
- [ ] DELETE
- [ ] PUT
- [ ] REMOVE
- [ ] MODIFY
- [ ] PATCH

EXERCISE: STANDARD METHODS 2

What is the difference between PUT and PATCH?

- [ ] with PATCH only the changed properties are transferred thus saving network traffic
- [ ] PUT creates a new object whereas PATCH updates an existing object
- [ ] there is no difference between PUT and PATCH

EXERCISE: INTERPRETATION (1)

Given the call

GET http://www.example.de/products/4711/producer

Which interpretation is right?

- [ ] get the producer of product with ID 4711
- [ ] read producer with ID 4711
- [ ] read product with ID 4711

EXERCISE: INTERPRETATION (2)

Given the call

PUT http://www.example.de/customer/4711
(body: customer object in JSON format)

Which interpretation is right?

- [ ] Update the customer with ID 4711 using the transferred data
- [ ] Create the customer with the ID 4711
- [ ] Set the ID of the customer to 4711

RESOURCE DATA REPRESENTATION

- Today, there are many common representations data objects - e.g. text, XML, HTML, JSON
- The client can use the 'Accept' header attribute as MIME-Type to request the result of a call in a certain format
- Today, servers often only support the JSON format, so it may well be that the desired format is not available!

Example 1: Customer with ID 1234 in XML format

```
GET /customer/1234
Host: beispiel.de
Accept: application/xml

Reply:
<customer>
    <nr>1234</nr>
    <name>Hermann Mann</name>
</customer>
```

Example 2: Customer with ID 1234 in JSON format

```
GET /customer/1234
Host: beispiel.de
Accept: application/json

Reply:
{
    "nr": 1234,
    "name": "Hermann Mann"
}
```

STATELESS COMMUNICATION

HTTP is a stateless protocol, i.e. every call is answered based on the data it contains and typically the data from other sources (database, files, other servers).

Advantages

- Failure protection through redundancy of similar servers
- Greater scalability through the use of similar server instances in a server cluster
- Lightweight: REST is lightweight in terms of the data exchanged between client and server. Using formats like JSON, REST APIs can be optimized for high performance.

Disadvantage

- For related requests some kind of common context must be restored - e.g. authenticated sessions

3.2 EXAMPLE: DESIGNING A REST API FOR ORDER MANAGEMENT

We will develop a Node.js server for order management to demonstrate the REST principles in practice.

Please download the fragment from the next page, it only implements a part of the API. A full solution is also available.



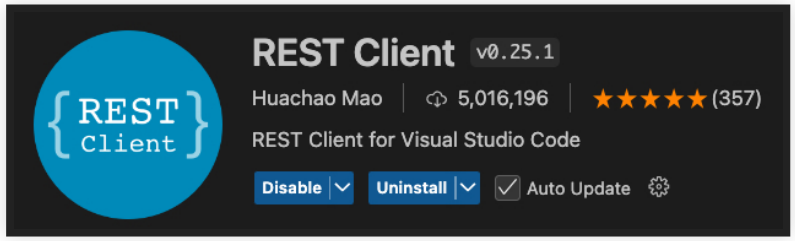
Server-Download:

Open the exercise project in folder `03-rest/order-server-exercise` (from Git repository). A solution can be found in `03-rest/order-server-solution`.

The file "README.md" tells you how to install and run the server. If viewed in VS Code preview mode (Ctrl+Shift+v) you will get a preview of the implemented API.

Testing your Server

No that the server is running you can test it using a tool like Postman. However, VS Code also offers a convenient plugin to test your REST client:



Steps for intial testing:

- install the plugin in VS Code
- open the tests in "test.http"
- execute the first test by clicking on "Send Request" after line 1. You should see a new window with the message "The order server is up and running!"
- execute the second test to get all orders



The file "test.http" contains developer tests for all API endpoints to verify that all API operations are basically working. Of course, in a professional development project you should test your server more thoroughly by coding unit tests for the operations.

Also, the operations are not implemented robustly, i.e. there are no checks whether the supplied inputs are valid.

The server implements the following data model (see folder "models"):



Remarks:

1. the above association between order and position is redundant, as the realising attribute "positions" exists at the same time.
2. the attribute "noPositions" is redundant, but is supplied to the client in order to be able to display the number of positions. The actual positions may be loaded in a second step.

...and offers the following interface to clients with "Order" being the main and "Position" the sub-resource:





According to the semantics of HTTP methods with REST, the server interface can be designed as a collection of endpoints as follows:

operation	HTTP method	path	request body	reply body	status codes
getOrders(): Order[]	GET	order		Order[]	200
getOrder(id: int): Order	GET	order/:id		Order	200, 404
createOrder(o: Order): Order	POST	order	Order	Order	201
updateOrder(o: Order): void	PUT	order	Order		200, 404
deleteOrder(id: int): void	DELETE	order/:id			200, 404
getPositions(orderId: int): Position[]	GET	order/:orderId/position		Position[]	200
createPosition(orderId: int, p: Position): void	POST	order/:orderId/position	Position		201
updatePosition(orderId: int, p: Position): void	PUT	order/:orderId/position	Position		200, 404
deletePosition(orderId: int, id: int): void	DELETE	order/:orderId/position/:id			200, 404

Syntax: given a path "a/:id/b".
Then :id is a placeholder for a string representing an id.

Design Rules

When designing the API endpoint (= server URL) for an operation, you typically apply the folowing rules:

- find a method: use the standard semantics for GET, POST, PUT, PATCH and DELETE
- path:
  - use the resource(s) the operation is dealing with (e.g. order)
  - when a resource can be identified by a numeric key, add it to the path (e.g. order id)
  - sub-resources become sub-paths (e.g. order/:id/position)
- request body: input parameters of object type must be sent in the request body
- reply body: here goes the operation result unless it can be expressed in the status code
- status code: use HTTP status codes, try to use the ones that match the desired semantics (e.g. 404 when an order id is not found)



Important: In order for the server design to work, every combination of method and path must be unique! Otherwise, the server cannot decide which operation to perform!

IMPLEMENTING THE SERVER API WITH NODE.JS AND EXPRESS

There are many ways to implement a REST API. Some popular examples:

Language	Platform	Remarks
Python	Flask, Django or FastAPI	FastAPI is very fast, generates Swagger API Documentation
Java	Spring Boot	ready-to-run application with embedded web server
C#	ASP.NET Core	great when integration with Microsoft Ecosystem

In this course we use Node.js and Express. Express is ideal for handling asynchronous operations (API requests) and due to the underlying JavaScript ecosystem there are many resources and tutorials available.

EXPRESS

There are many express tutorials available, e.g. this one. To get acquainted with express, we choose a different approach: since you already have the order server running, we can examine its code step by step.



ORDER SERVER: BASICS

The server project is a typical Node.js project based on npm.

- All source code is under "src"
- Configuration information is specified in file "./.env" (e.g. PORT). See "src/server.ts" (server entry file) on how the npm package dotenv is used to read the server PORT.



Overview of the project structure:

file/folder	description
src/server.ts	server main file - express setup, defines routes, starts listener
src/routes.ts	tells express which routes to listen for
src/controllers	contains endpoint services grouped by resource. Here: orderController
src/database	folder were you put your database code in. Here: data.ts - just an array of orders and functions for data access by controllers
src/models	classes for the data objects (here: Order, Position)

DEFINING THE ROUTES

For maintainability reasons the route definition is separate from the implementation of the endpoint operations. Thus "routes.ts" defines the routes with express and references the corresponding functions in "orderController.ts":

```
const defineRoutes = (app: express.Express) => {
  app.get('/api/order', getOrders);
  app.get('/api/order/:id', getOrderById);
  app.post('/api/order', postOrder);
  app.put('/api/order', putOrder);
  app.delete('/api/order/:id', deleteOrder);

  app.get('/api/order/:orderId/position', getPositions);
  app.post('/api/order/:orderId/position', postPosition);
  app.put('/api/order/:orderId/position', putPosition);
  app.delete('/api/order/:orderId/position/:positionNo', deletePosition);
};
```



ENDPOINT IMPLEMENTATION

Each function implementing an endpoint receives two parameters:

- a Request object describing the request. This is used to get parameters and body data
- a Response object which is used to setup the response message to the client

The next page explains how the operation

```
createPosition(orderId: int, p: Position): void
```

is implemented. Please also inspect the other endpoint implementations.

Two inputs are read from the request before the operation can be performed:

```
const postPosition = (req: Request, res: Response) => {
  try {
    const orderId = Number(req.params.orderId);
    const body = req.body;
    const ok = createPosition(orderId, body);

    if (ok) {
      res.status(201).send();
    } else {
      res.status(400).send();
    }
  } catch (err) {
    res.status(400).send();
  }
};
```

- the orderId is read from the path
- the Position object is read from the request body
- if the position could be created (nr must be unique!), the reply status will be "201"
- otherwise, "400" is sent back

3.3 DISCUSSION

Every approach or technique has advantages and disadvantages depending on the purpose of use and context. In this section we discuss the pros and cons of using REST for distributed systems.

Advantages of REST APIs are:

1. Statelessness
    - Scalability: the server does not maintain client states between requests
    - Fault tolerance: a crashed server can easily be replaced by another one
2. Simplicity:
    - HTTP is widely used and well known
    - easy to understand readable URLs
3. Flexibility:
    - HTTP is widely available and can be used by any client (mobile, web, desktop, etc.)
    - Various Data Formats: mostly JSON but others also possible (XML, HTML, plain text)
4. Caching: builtin HTTP-mechanism using headers

Disadvantages:

1. Overhead
    - Stateless nature: each request must carry all necessary information (like authentication tokens), which can increase network traffic and payload size
    - Redundant Data Transfer: more data than necessary maybe transferred
2. Performance Limitations
    - HTTP and JSON are not the most efficient in terms of performance
    - Heavy payloads: JSON can add significant overhead compared to more compact formats like Protocol Buffers (Protobuf) or MessagePack
3. No Built-in State Management
4. Overloading of HTTP Methods: often ambiguity in how HTTP methods should be applied
5. Documentation: tools like Swagger and OpenAPI help, but no self-explanatory contracts like GraphQL



When to Use REST APIs?

Best suited for:

- Web applications
- CRUD operations
- microservices
- mobile app backends

Less suited for:

- Real-time communication
- applications with complex querying requirements
- scenarios requiring low-latency communication.

Conclusion

- great for Web applications
- for complex queries or real-time updates: use GraphQL or WebSockets

3.4 EXERCISE: EXTENDING THE ORDER SERVER

Add the following operation to the order server:

```
public searchOrders(searchTerm: String): Order[];
```

"searchOrders" should return an array of all orders which contain "searchTerm" in at least one of these properties:

- order: customer, date
- position: number, article, amount

Test your implementation by adding data to "data.ts" and by adding calls in "test.http".

3.5 EXERCISE: DEVELOPING A NODE.JS REST API FOR INVOICE AND CUSTOMER MANAGEMENT

Our final exercise is about developing a more complex REST API for invoice and customer management. We will use the server later with our Angular and NextJS clients.



You can choose between a ready solution and examine it or download the fragment (button "Exercise") to improve your development skills.

Open the exercise project in folder `03-rest/acme-server-exercise` (from Git repository). A solution can be found in `03-rest/acme-server-solution`.

See file "README.md" for instructions.

UNDERLYING DATA MODEL



Class InvoiceForCustomer is a so-called view model for clients that need to access an invoice together with the associated customer information in one server call.

RESTFUL SERVER API

Part I: retrieve user and revenue data

Operation	Method	URL	request body	response body	http status code	remark
getUser(email: string): User	GET	/user?email=:email		User	200, 404	
getRevenues(): Revenue[]	GET	/revenue		Revenue[]	200	

Part II: retrieve invoice data

Operation	Method	URL	request body	response body	ht sta co
getInvoiceById(id: int): Invoice	GET	/invoice?id=:id		Invoice	20 40
getLatestInvoices(limit: int): Invoices[]	GET	/invoice/latest/:limit		Invoice[]	20
getFilteredInvoices(query: string, currentPage: int): Invoices[]	GET	/invoice/filtered?query=&currentPage=		Invoice[]	20

Part III: create, update and delete invoices

Operation	Method	URL	request body	response body	http status code	remark
createInvoice(invoice: Invoice): int	POST	/invoice	Invoice		201	result is id
updateInvoice(invoice: Invoice): void	PUT	/invoice/:id	Invoice		200, 404	
deleteInvoice(id: int): void	DELETE	/invoice/:id			200, 404	

DATA STORAGE AND ACCESS

Instead of using a database (the way to go in a real application!) we just create all data in memory and use maps to access the data (see src/Database/Data.ts):

Map	Key	Usage example
users	email	const u: User = Data.users.get('user@mail.com');
customers	id	const result: Customer[] = []; Data.customers.forEach(c => result.push(c));
revenue	month	const result: Revenue[] = []; Data.revenues.forEach(r => result.push(r));
invoices	id	const invoice = Data.invoices.get(id);

STEP 1: DEVELOP THE CUSTOMERCONTROLLER

Code the empty method in the controller and verify that it works by running the first test in "tests.http"! You should receive the following output:

```
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 832
ETag: W/"340-A//GQH1whqdmhaAG4JMyQf5r0QI"
Date: Fri, 20 Sep 2024 15:11:13 GMT
Connection: keep-alive
keep-alive: timeout=5

[
  {
    "id": "d6e15727-9fe1-4961-8c5b-ea44a9bd81aa",
    "name": "Evil Rabbit",
    "email": "evil@rabbit.com",
    "image_url": "/customers/evil-rabbit.png"
  },
  ...
```

STEP 2: DEVELOP THE USERCONTROLLER

Code the empty method in the controller and verify that it works by running the second test in "tests.http"! You should receive the following output:

```
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 103
ETag: W/"67-caz9q9JY33DLC7Is1WCRjN9JAQ4"
Date: Fri, 20 Sep 2024 15:17:03 GMT
Connection: keep-alive
keep-alive: timeout=5

{
  "id": "410544b2-4001-4271-9855-fec4b6a6442a",
  "name": "User",
  "email": "user@mail.com",
  "password": "123456"
}
```

STEP 3: DEVELOP THE REVENUECONTROLLER

Code the empty method in the controller and verify that it works by running the third test in "tests.http"! You should receive the following output:

```
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 373
ETag: W/"175-8NPwSA+LtepAWibAyvXEI8+9mC8"
Date: Fri, 20 Sep 2024 15:18:23 GMT
Connection: keep-alive
keep-alive: timeout=5

[
  {
    "month": "Jan",
    "revenue": 2000
  },
  ...
```



STEP 4: DEVELOP THE INVOICECONTROLLER

Look at the method headers in the InvoiceController class and develop the methods one-by-one. Use the tests provided to verify that your code works correctly.



